

RHEL / RHCSA STUDY & PRACTICAL LAB GUIDE

Package Management | Repo Server & Repo Client
Network Configuration | FirewallD | SELinux

Definitions • Use Cases • Step-by-Step Commands • Hands-on Labs • Cheat Sheets

From Basics to Advanced Level

Table of Contents

1. Package Management (RPM & DNF)
2. Repo Server & Repo Client Configuration
3. Network Configuration (NetworkManager / nmcli)
4. Firewall (firewalld)
5. SELinux
6. Master Cheat Sheet (All Commands, One Page)
7. Exam-Style Quick Answers (Q&A Bank)

How to use this guide: Each topic follows the same structure — Definition, Use Case, Key Concepts, Step-by-Step Commands (basic to advanced), a Hands-on Lab you can replicate on your RHEL VM or ESXi homelab, and a Cheat Sheet table for quick revision before the RHCSA exam or an interview.

Package Management

RPM & DNF - Installing, Removing, Updating and Querying Software

1.1 Definition

Package management is the process of installing, updating, configuring, querying and removing software on a Linux system in a controlled, dependency-aware way. On RHEL-family systems (RHEL, CentOS, Rocky, Alma, Fedora) software is distributed as **.rpm** (Red Hat Package Manager) files. **RPM** is the low-level tool that installs/queries individual .rpm files, while **DNF** (Dandified YUM, the successor to YUM) is the high-level tool that resolves dependencies automatically by talking to configured repositories.

1.2 Use Case

- Installing server software (httpd, mariadb-server, ansible-core, podman, etc.) without manually chasing dependencies.
- Keeping a fleet of servers patched and secure (security errata, CVE fixes) via `dnf update`.
- Rolling back a bad update using DNF history/transactions.
- Auditing what is installed, which package owns a file, or verifying package integrity (useful in security audits).
- Managing versioned application streams (e.g. multiple PHP/Python versions) with DNF modules.

1.3 Key Concepts

- **RPM** = works on a single local .rpm file, no dependency resolution, no internet needed.
- **DNF** = works against repositories (local or remote), resolves and downloads dependencies automatically.
- **Repository (repo)** = a location (HTTP/FTP/local path) containing packages + metadata that DNF searches.
- **Transaction history** = every dnf install/remove/update is logged and can be undone.
- **Module streams** = DNF can install specific versions of an application (e.g. `postgresql:15`).
- **GPG signing** = packages are verified against a public key so tampered packages are rejected.

1.4 How Many Steps to Achieve Common Tasks

Below is the realistic step count for each common package-management task on the exam or in production.

Task	Steps Needed	Notes
Install a package	1 step	<code>dnf install -y <pkg></code> (repo must already be configured)
Remove a package	1 step	<code>dnf remove -y <pkg></code> (add <code>--noautoremove</code> to keep deps)
Install from a local .rpm	2 steps	Copy file to host, then <code>dnf install ./file.rpm</code>
Rollback an update	2 steps	<code>dnf history list</code> , then <code>dnf history undo <id></code>
Verify package integrity	1 step	<code>rpm -V <package></code>
Find which package owns a file	1 step	<code>rpm -qf /path/to/file</code>
Install a package group	2 steps	<code>dnf group list</code> , then <code>dnf group install "Group Name"</code>
Enable a module stream	2 steps	<code>dnf module list <name></code> , then <code>dnf module enable <name>:<stream></code>

1.5 Step-by-Step Commands - Basic to Advanced

A. Basic - Query & Search

```
# Search for a package by name/keyword
dnf search httpd

# Get detailed info about a package
dnf info httpd

# List all installed packages
dnf list installed

# List only available (not installed) packages
dnf list available

# Check which package provides a command/file
dnf provides /usr/sbin/httpd
```

B. Basic - Install / Remove

```
# Install a package (auto-confirm with -y)
dnf install -y httpd

# Install multiple packages at once
dnf install -y httpd mariadb-server php

# Remove a package
dnf remove -y httpd

# Reinstall a package (fix corrupted files)
dnf reinstall -y httpd
```

C. Intermediate - Update & Groups

```
# Check for available updates
dnf check-update

# Update everything on the system
dnf update -y

# Update a single package
dnf update -y kernel

# List package groups (e.g. 'Server with GUI')
dnf group list

# Install an entire group
dnf group install -y "Development Tools"
```

D. Intermediate - RPM Low-Level Queries

```
# List all files installed by a package
rpm -ql httpd

# Show package that owns a specific file
rpm -qf /etc/httpd/conf/httpd.conf

# Show config files only
rpm -qc httpd
```

```
# Verify installed files against package DB (detect tampering)
rpm -V httpd

# Install a downloaded/local rpm and resolve deps via dnf
dnf install -y ./custom-tool-1.2.rpm
```

E. Advanced - History, Rollback & Modules

```
# View all past dnf transactions
dnf history list

# View details of one transaction
dnf history info 27

# Undo (rollback) a transaction
dnf history undo 27

# Redo a transaction
dnf history redo 27

# List available module streams (e.g. multiple app versions)
dnf module list postgresql

# Enable a specific stream then install
dnf module enable -y postgresql:15
dnf install -y postgresql-server

# Download a package + all deps without installing (offline use)
dnf download --resolve --destdir=/tmp/pkgs httpd

# Clean cached metadata / rebuild cache
dnf clean all
dnf makecache
```

HANDS-ON LAB: Package Management on your RHEL VM

1. SSH into your RHEL 9 VM: `ssh root@<vm-ip>`
2. Search and install nginx-like web server: `dnf install -y httpd`
3. Confirm it installed and check owned files: `rpm -ql httpd | head`
4. Simulate a bad update, then rollback: `dnf history list` then `dnf history undo <id>`
5. Remove the package cleanly: `dnf remove -y httpd`
6. Verify it is gone: `rpm -q httpd` (should say 'package httpd is not installed')

1.6 Cheat Sheet - Package Management

Command	Purpose	Example
<code>dnf install</code>	Install package(s)	<code>dnf install -y vim</code>
<code>dnf remove</code>	Remove package(s)	<code>dnf remove -y vim</code>
<code>dnf update</code>	Update all/one package	<code>dnf update -y</code>
<code>dnf search</code>	Search by keyword	<code>dnf search editor</code>
<code>dnf info</code>	Show package details	<code>dnf info vim</code>

Command	Purpose	Example
<code>dnf list installed</code>	List installed pkgs	<code>dnf list installed grep ssh</code>
<code>dnf provides</code>	Find owning package	<code>dnf provides /usr/bin/vim</code>
<code>dnf history</code>	Show/undo transactions	<code>dnf history undo 12</code>
<code>dnf group list/install</code>	Manage package groups	<code>dnf group install -y 'Base'</code>
<code>dnf module list/enable</code>	Manage app-stream versions	<code>dnf module enable php:8.2</code>
<code>rpm -q</code>	Query installed package	<code>rpm -q httpd</code>
<code>rpm -qi</code>	Detailed package info	<code>rpm -qi httpd</code>
<code>rpm -ql</code>	List files in package	<code>rpm -ql httpd</code>
<code>rpm -qf</code>	Find package owning file	<code>rpm -qf /etc/passwd</code>
<code>rpm -V</code>	Verify package integrity	<code>rpm -V httpd</code>
<code>rpm -ivh</code>	Install local rpm (no deps)	<code>rpm -ivh pkg.rpm</code>
<code>rpm -e</code>	Erase/remove via rpm	<code>rpm -e pkg</code>

Repo Server & Repo Client

Building and Consuming a Custom Local/Remote YUM-DNF Repository

2.1 Definition

A **Repo Server** is a machine that hosts RPM packages plus repository metadata (generated by `createrepo_c`) and serves them over HTTP, FTP, or a local/NFS path. A **Repo Client** is any machine that points to that server through a `.repo` file in `/etc/yum.repos.d/` so that `dnf` can install packages from it. This is exactly the pattern behind tools like Foreman/Katello, Satellite, and your own offline homelab mirror.

2.2 Use Case

- Air-gapped / offline environments with no internet access to Red Hat CDN.
- Faster, bandwidth-saving internal mirror for a data center or homelab (download once, serve many).
- Version pinning and change control - only approved packages are made available to clients.
- Serving custom in-house built RPMs (internal tools, patched packages).
- Foundation of patch management systems (Foreman/Katello, Spacewalk, Satellite) - relevant to your Prodevans internship work with Foreman/Katello.

2.3 Key Concepts

- **createrepo_c** = scans a directory of `.rpm` files and builds the `repodata/` metadata DNF needs.
- **baseurl** = the URL/path in a `.repo` file telling the client where the repo lives.
- **gpgcheck** = whether the client verifies package signatures (0 = off, 1 = on).
- **reposync** = downloads (mirrors) an entire remote repo locally, package files + metadata.
- **httpd (Apache)** = commonly used to expose the repo directory over HTTP for remote clients.
- **SELinux context** = repo directories served by `httpd` need the `httpd_sys_content_t` context (see Topic 5).

2.4 How Many Steps to Achieve This

Task	Steps Needed	Notes
Create a local (<code>file:///</code>) repo	3 steps	Install <code>createrepo_c</code> , put RPMs in a dir, run <code>createrepo_c</code>
Serve repo over HTTP	6 steps	Install <code>httpd</code> , place repo under <code>/var/www/html</code> , fix SELinux+firewall, start <code>httpd</code>
Point a client at it	2 steps	Create <code>.repo</code> file, <code>dnf makecache</code>
Mirror a remote repo locally (<code>reposync</code>)	3 steps	Install <code>dnf-plugins-core</code> , <code>reposync</code> , <code>createrepo_c</code>
Update repo after adding new RPMs	1 step	<code>createrepo_c --update /path/to/repo</code>

2.5 Step-by-Step Commands - Basic to Advanced

A. Basic - Build a Local Repository (SERVER side)

```
# 1. Install the repo creation tool
dnf install -y createrepo_c
```

```
# 2. Create a directory and drop your RPM files in it
mkdir -p /repo/myrepo
cp /path/to/*.rpm /repo/myrepo/

# 3. Generate repodata/ metadata
createrepo_c /repo/myrepo

# 4. (Client, same machine) create a .repo pointer file
cat > /etc/yum.repos.d/myrepo.repo << 'EOF'
[myrepo]
name=My Local Repository
baseurl=file:///repo/myrepo
enabled=1
gpgcheck=0
EOF

# 5. Refresh cache and test
dnf clean all && dnf makecache
dnf list --disablerepo='*' --enablerepo='myrepo'
```

B. Intermediate - Serve the Repo over HTTP

```
# Install and enable Apache
dnf install -y httpd
systemctl enable --now httpd

# Move / symlink your repo under the Apache document root
mkdir -p /var/www/html/repo
cp -r /repo/myrepo/* /var/www/html/repo/

# Fix SELinux context so httpd is allowed to read it
restorecon -Rv /var/www/html/repo

# Open the firewall for HTTP
firewall-cmd --permanent --add-service=http
firewall-cmd --reload

# Test locally
curl -I http://localhost/repo/repodata/repomd.xml
```

C. Intermediate - Repo Client (REMOTE machine)

```
# On the client, point to the server's HTTP repo
cat > /etc/yum.repos.d/myrepo.repo << 'EOF'
[myrepo]
name=Internal Repo Server
baseurl=http://<repo-server-ip>/repo
enabled=1
gpgcheck=0
EOF

# Refresh and install from it
dnf clean all && dnf makecache
```



```
dnf install -y <package-from-that-repo>
```

D. Advanced - Mirror an Upstream Repo with reposync

```
# Install the reposync plugin
dnf install -y dnf-plugins-core

# Mirror an entire enabled remote repo locally
reposync --repoid=baseos --download-path=/repo/mirror -m

# Build metadata for the mirrored copy
createrepo_c /repo/mirror/baseos

# Keep it updated later (only pulls new/changed rpms)
reposync --repoid=baseos --download-path=/repo/mirror -m --newest-only
createrepo_c --update /repo/mirror/baseos
```

E. Advanced - GPG-Signed Repo (Production Pattern)

```
# Sign packages before adding, then reference the key on clients
rpm --import http://<repo-server-ip>/RPM-GPG-KEY-myorg

# .repo file with signing enforced
cat > /etc/yum.repos.d/myrepo.repo << 'EOF'
[myrepo]
name=Signed Internal Repo
baseurl=http://<repo-server-ip>/repo
enabled=1
gpgcheck=1
gpgkey=http://<repo-server-ip>/RPM-GPG-KEY-myorg
EOF
```

HANDS-ON LAB: Build a Repo Server + Client Pair on Your Homelab

1. On **server VM**: install createrepo_c and httpd, create /var/www/html/repo, copy a few RPMs in.
2. Run createrepo_c /var/www/html/repo then restorecon -Rv /var/www/html/repo.
3. Open firewall: firewall-cmd --permanent --add-service=http && firewall-cmd --reload.
4. Verify from the server itself with curl -I http://localhost/repo/repodata/repomd.xml.
5. On a **second (client) VM**: create /etc/yum.repos.d/myrepo.repo pointing baseurl at the server's IP.
6. Run dnf clean all && dnf makecache then install one of the mirrored packages to confirm end-to-end.

2.6 Cheat Sheet - Repo Server / Client

Command / File	Purpose	Example
createrepo_c <dir>	Build repo metadata	createrepo_c /repo/myrepo
createrepo_c --update	Refresh metadata after adding RPMs	createrepo_c --update /repo/myrepo
/etc/yum.repos.d/*.repo	Client repo definition file	baseurl, enabled, gpgcheck
dnf repolist	Show configured repos	dnf repolist all

Command / File	Purpose	Example
<code>dnf clean all</code>	Clear DNF cache	<code>dnf clean all</code>
<code>dnf makecache</code>	Rebuild metadata cache	<code>dnf makecache</code>
<code>reposync</code>	Mirror a remote repo locally	<code>reposync --repoid=baseos -m</code>
<code>restorecon -Rv</code>	Fix SELinux labels on repo dir	<code>restorecon -Rv /var/www/html/repo</code>
<code>firewall-cmd --add-service=http</code>	Allow HTTP through firewall	<code>firewall-cmd --permanent --add-service=http</code>
<code>rpm --import</code>	Trust a GPG signing key	<code>rpm --import http://server/RPM-GPG-KEY</code>

Network Configuration

NetworkManager, nmcli, nmtui - IP, DNS, Hostname & Bonding

3.1 Definition

Network configuration on RHEL 8/9 is managed by **NetworkManager**, a daemon that controls network devices and connections. You interact with it through **nmcli** (command-line), **nmtui** (text UI), or the Cockpit web console. A 'connection' is a saved profile (IP, gateway, DNS, etc.) that can be applied to a physical or virtual network 'device' (interface).

3.2 Use Case

- Assigning a static IP to a server so services/DNS entries remain stable (critical for repo servers, DNS, HAProxy nodes).
- Switching an interface between DHCP and static addressing during provisioning.
- Setting custom DNS resolvers and a search domain for internal name resolution.
- Creating bonded/teamed interfaces for network redundancy on production hosts.
- Diagnosing connectivity issues (routing, DNS, link state) during troubleshooting.
- Setting up Tailscale/VPN routing on top of the base NIC configuration.

3.3 Key Concepts

- **Device** = physical/virtual NIC, e.g. ens160, eth0.
- **Connection profile** = saved configuration (name can differ from the device name), stored under `/etc/NetworkManager/system-connections/`.
- **ipv4.method** = manual (static) or auto (DHCP).
- **Gateway** = next-hop router for traffic leaving the subnet.
- **Hostname** = managed separately via `hostnamectl`, independent of IP config.
- **/etc/hosts** = local static name resolution, checked before DNS.

3.4 How Many Steps to Achieve Common Tasks

Task	Steps Needed	Notes
Set a static IP	3 steps	nmcli con mod (address+gateway+dns), then con up
Switch back to DHCP	2 steps	nmcli con mod ipv4.method auto, then con up
Change hostname	1 step	hostnamectl set-hostname
Add a secondary IP	2 steps	nmcli con mod +ipv4.addresses, con up
Create a bond of 2 NICs	4 steps	Create bond, add slaves, set IP, bring up
Add a static route	2 steps	nmcli con mod +ipv4.routes, con up

3.5 Step-by-Step Commands - Basic to Advanced

A. Basic - Viewing Network State

```
# Show all devices and their state
nmcli device status
```

```
# Show all connection profiles
nmcli connection show

# Show IP addressing (classic tool, still very common)
ip addr show
ip a

# Show routing table
ip route show

# Current hostname
hostnamectl status
```

B. Basic - Static IP Configuration

```
# Set a static IPv4 address + prefix on connection 'ens160'
nmcli con mod ens160 ipv4.addresses 192.168.50.20/24

# Set the default gateway
nmcli con mod ens160 ipv4.gateway 192.168.50.1

# Set DNS servers
nmcli con mod ens160 ipv4.dns "8.8.8.8 1.1.1.1"

# Switch the method from DHCP to manual (static)
nmcli con mod ens160 ipv4.method manual

# Apply the changes by bringing the connection up again
nmcli con up ens160
```

C. Intermediate - Hostname & DNS

```
# Set the system hostname permanently
hostnamectl set-hostname repo-server.lab.local

# Add a manual entry to /etc/hosts
echo "192.168.50.20 repo-server.lab.local repo-server" >> /etc/hosts

# Test name resolution
getent hosts repo-server.lab.local
dig repo-server.lab.local
```

D. Intermediate - New Connection Profiles

```
# Create a brand-new static connection profile from scratch
nmcli con add type ethernet con-name static-lan ifname ens192 \
ipv4.addresses 192.168.50.30/24 ipv4.gateway 192.168.50.1 \
ipv4.dns "1.1.1.1" ipv4.method manual

# Activate it
nmcli con up static-lan

# Delete a connection profile
nmcli con delete static-lan
```

E. Advanced - Bonding, Routes & Troubleshooting

```
# Create an active-backup bond for redundancy
nmcli con add type bond con-name bond0 ifname bond0 mode active-backup
nmcli con add type ethernet ifname ens160 master bond0
nmcli con add type ethernet ifname ens192 master bond0
nmcli con mod bond0 ipv4.addresses 192.168.50.40/24 ipv4.method manual
nmcli con up bond0

# Add an extra static route (e.g. to reach another subnet)
nmcli con mod ens160 +ipv4.routes "10.10.0.0/24 192.168.50.1"
nmcli con up ens160

# Live troubleshooting commands
ss -tulnp # listening ports
nmcli general logging # NM log level
journalctl -u NetworkManager -f
nmcli con show ens160 # full profile dump
```

HANDS-ON LAB: Configure a Static IP for your Homelab ESXi Guest

1. Check current device/connection name: `nmcli device status`.
2. Set a static IP inside your homelab subnet: `nmcli con mod ens160 ipv4.addresses 192.168.50.50/24`.
3. Set the gateway and DNS: `ipv4.gateway 192.168.50.1` and `ipv4.dns "1.1.1.1"`.
4. Switch to manual and apply: `ipv4.method manual` then `nmcli con up ens160`.
5. Verify with `ip a` and confirm connectivity with `ping -c3 192.168.50.1`.
6. Set a matching hostname with `hostnamectl set-hostname` and confirm with `hostnamectl status`.

3.6 Cheat Sheet - Network

Command	Purpose	Example
<code>nmcli device status</code>	List interfaces + state	<code>nmcli device status</code>
<code>nmcli con show</code>	List connection profiles	<code>nmcli con show --active</code>
<code>nmcli con mod</code>	Modify a connection setting	<code>nmcli con mod eth0 ipv4.method manual</code>
<code>nmcli con up / down</code>	Activate / deactivate profile	<code>nmcli con up eth0</code>
<code>nmcli con add</code>	Create a new profile	<code>nmcli con add type ethernet ...</code>
<code>ip addr / ip a</code>	Show IP addresses	<code>ip a</code>
<code>ip route</code>	Show routing table	<code>ip route show</code>
<code>hostnamectl</code>	View/set hostname	<code>hostnamectl set-hostname web01</code>
<code>nmtui</code>	Text-based network UI	<code>nmtui</code>
<code>dig / getent hosts</code>	DNS lookups	<code>dig example.com</code>
<code>ss -tulnp</code>	Show listening sockets	<code>ss -tulnp</code>
<code>nm-connection-editor</code>	GUI editor (if desktop present)	<code>nm-connection-editor</code>

Firewall

firewalld - Zones, Services, Ports & Rich Rules

4.1 Definition

firewalld is RHEL's dynamic firewall management daemon, built on top of nftables (previously iptables). Instead of writing raw packet-filter rules, you work with human-readable **zones** (trust levels like public, internal, dmz) and attach **services** or **ports** to them. Changes can be applied live at **runtime** and optionally saved **permanently**.

4.2 Use Case

- Opening only the ports a server actually needs (e.g. 80/443 for a web server, 22 for SSH) - least privilege.
- Segmenting trust levels: a public-facing NIC vs. an internal management NIC using different zones.
- Allowing access only from specific source IP ranges (e.g. management subnet only) with rich rules.
- Quickly locking down a compromised host using panic mode.
- Exposing your homelab repo server / OpenShift API only to trusted addresses.

4.3 Key Concepts

- **Zone** = a named set of firewall rules representing a trust level (default zone is usually 'public').
- **Service** = a pre-defined bundle of ports/protocols (e.g. 'http' = tcp/80).
- **Runtime vs Permanent** = --permanent writes to config but needs --reload (or a re-add without --permanent) to also apply now; without --permanent changes are runtime-only and lost on reboot/reload.
- **Rich rule** = a more expressive rule (source IP + service/port + action) beyond simple service/port add.
- **Masquerade** = NAT, needed when the host is also acting as a router/gateway.
- **Panic mode** = instantly drops all traffic, used in an active-incident lockdown.

4.4 How Many Steps to Achieve Common Tasks

Task	Steps Needed	Notes
Open a named service (e.g. http)	2 steps	add-service --permanent, then reload
Open a custom port	2 steps	add-port --permanent, then reload
Allow only one source IP to a port	2 steps	add-rich-rule --permanent, then reload
Change the default zone	1 step	firewall-cmd --set-default-zone=internal
Assign an interface to a zone	2 steps	add-interface --permanent, then reload
Emergency lockdown	1 step	firewall-cmd --panic-on

4.5 Step-by-Step Commands - Basic to Advanced

A. Basic - Status & Inspection

```
# Is firewalld running?
systemctl status firewalld

# General state
firewall-cmd --state
```

```
# List all zones
firewall-cmd --get-zones

# Show the default zone
firewall-cmd --get-default-zone

# Show full config of the active zone
firewall-cmd --list-all
```

B. Basic - Opening Services & Ports

```
# Allow HTTP permanently
firewall-cmd --permanent --add-service=http

# Allow a custom TCP port (e.g. app on 8080)
firewall-cmd --permanent --add-port=8080/tcp

# Apply the permanent changes now
firewall-cmd --reload

# Verify
firewall-cmd --list-services
firewall-cmd --list-ports
```

C. Intermediate - Removing Rules & Zones

```
# Remove a service
firewall-cmd --permanent --remove-service=http
firewall-cmd --reload

# List every service firewalld knows about
firewall-cmd --get-services

# Assign a NIC to the 'internal' (more trusted) zone
firewall-cmd --permanent --zone=internal --change-interface=ens192
firewall-cmd --reload

# Change the system default zone
firewall-cmd --set-default-zone=internal
```

D. Advanced - Rich Rules (Source-Restricted Access)

```
# Allow SSH only from the management subnet
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
source address="192.168.50.0/24" service name="ssh" accept'
firewall-cmd --reload

# Explicitly reject (not just drop) traffic from a bad IP
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
source address="203.0.113.5" reject'
firewall-cmd --reload

# List all rich rules currently configured
firewall-cmd --list-rich-rules
```

E. Advanced - NAT, Port Forwarding & Panic Mode

```
# Enable masquerading (NAT) - needed if host routes traffic for others
firewall-cmd --permanent --add-masquerade
firewall-cmd --reload

# Forward incoming 8443 to internal service on port 443
firewall-cmd --permanent --add-forward-port=port=8443:proto=tcp:toport=443
firewall-cmd --reload

# EMERGENCY: drop all traffic instantly
firewall-cmd --panic-on

# Restore normal operation
firewall-cmd --panic-off
```

HANDS-ON LAB: Secure your Repo Server with firewalld

1. Check current firewall state: `firewall-cmd --state` and `firewall-cmd --list-all`.
2. Open HTTP for the repo you built in Topic 2: `firewall-cmd --permanent --add-service=http`.
3. Restrict SSH to your home/office subnet only using a rich rule (see section 4.5-D).
4. Reload and verify: `firewall-cmd --reload && firewall-cmd --list-all`.
5. Test from another host: `curl -I http://<server-ip>/repo/repodata/repomd.xml`.
6. Practice panic mode, then turn it back off: `firewall-cmd --panic-on` then `--panic-off`.

4.6 Cheat Sheet - Firewalld

Command	Purpose	Example
<code>firewall-cmd --state</code>	Check if firewalld is active	<code>firewall-cmd --state</code>
<code>firewall-cmd --list-all</code>	Show full active-zone config	<code>firewall-cmd --list-all</code>
<code>--add-service / --remove-service</code>	Allow/deny a named service	<code>--add-service=https</code>
<code>--add-port / --remove-port</code>	Allow/deny a raw port	<code>--add-port=8080/tcp</code>
<code>--add-rich-rule</code>	Advanced source/action rule	<code>source address=... accept</code>
<code>--permanent</code>	Persist rule across reboot	<code>--permanent --add-service=ssh</code>
<code>--reload</code>	Apply permanent rules now	<code>firewall-cmd --reload</code>
<code>--get-zones / --get-services</code>	List available zones/services	<code>firewall-cmd --get-zones</code>
<code>--set-default-zone</code>	Change default zone	<code>--set-default-zone=internal</code>
<code>--add-masquerade</code>	Enable NAT	<code>firewall-cmd --add-masquerade</code>
<code>--add-forward-port</code>	Port forwarding rule	<code>port=8443:toport=443</code>
<code>--panic-on / --panic-off</code>	Emergency drop-all / restore	<code>firewall-cmd --panic-on</code>

SELinux

Security-Enhanced Linux - Modes, Contexts, Booleans & Troubleshooting

5.1 Definition

SELinux (Security-Enhanced Linux) is a Mandatory Access Control (MAC) layer built into the kernel. Where normal Linux permissions (rwx) are Discretionary Access Control, SELinux adds a second, independent layer of enforcement based on **security contexts** (labels) attached to every process and file, checked against a loaded **policy**. Even root can be blocked by SELinux if an action isn't allowed by policy.

5.2 Use Case

- Containing a compromised service (e.g. a hacked web server process still can't read /etc/shadow).
- Enforcing that services only touch the files/ports they are explicitly allowed to (defense in depth).
- Required for RHCSA/RHCE exams and for any production RHEL security-compliance environment (STIG, PCI-DSS).
- Diagnosing 'Permission denied' errors on services (like httpd serving your repo in Topic 2) that are actually SELinux denials, not standard Unix permission problems.
- Allowing custom app ports/booleans in a controlled, auditable way instead of disabling SELinux entirely.

5.3 Key Concepts

- **Modes**: Enforcing (blocks + logs violations), Permissive (only logs, doesn't block), Disabled (off entirely).
- **Context format**: user:role:type:level, e.g. system_u:object_r:httpd_sys_content_t:s0 - the **type** field is what matters most day to day (Type Enforcement).
- **Domain** = the type assigned to a running process (e.g. httpd_t for the Apache process).
- **Boolean** = an on/off switch that toggles a chunk of policy without writing custom rules (e.g. httpd_can_network_connect).
- **Port labeling** = SELinux tracks which ports a service type is allowed to bind to via semanage port.
- **Audit log** = denials are recorded in /var/log/audit/audit.log, readable via ausearch/audit2why.

5.4 How Many Steps to Achieve Common Tasks

Task	Steps Needed	Notes
Check current mode	1 step	getenforce
Temporarily switch mode (till reboot)	1 step	setenforce 0 / setenforce 1
Permanently change mode	2 steps	Edit /etc/selinux/config, then reboot
Fix a mislabeled file/dir	1 step	restorecon -Rv /path
Set a custom persistent context	2 steps	semanage fcontext -a, then restorecon
Allow a non-standard port for a service	1 step	semanage port -a -t <type> -p tcp <port>
Toggle a boolean	1 step	setsebool -P <boolean> on
Diagnose a denial	2-3 steps	ausearch, audit2why / audit2allow

5.5 Step-by-Step Commands - Basic to Advanced

A. Basic - Checking Status & Mode

```
# Quick current mode (Enforcing/Permissive/Disabled)
getenforce

# Full status: mode, policy type, mount point, etc.
sestatus

# Set mode temporarily (lost at reboot)
setenforce 0 # Permissive
setenforce 1 # Enforcing
```

B. Basic - Viewing Contexts

```
# Show SELinux context of files (ls -Z)
ls -Z /var/www/html

# Show SELinux context of running processes (ps -Z)
ps -eZ | grep httpd

# Show your own login context
id -Z
```

C. Intermediate - Permanent Mode Change

```
# Edit the config file for a persistent mode change
vi /etc/selinux/config

# SELINUX=enforcing <- change this line (enforcing/permissive/disabled)

# A reboot is required for Disabled <-> Enforcing transitions
reboot
```

D. Intermediate - Fixing File Contexts

```
# Restore default context recursively (most common fix)
restorecon -Rv /var/www/html/repo

# Set a one-off context manually (temporary, lost on relabel)
chcon -t httpd_sys_content_t /var/www/html/repo -R

# Set a PERSISTENT custom rule so restorecon always applies it correctly
semanage fcontext -a -t httpd_sys_content_t "/data/webcontent(/.*)?"
restorecon -Rv /data/webcontent
```

E. Advanced - Booleans, Ports & Denial Troubleshooting

```
# List booleans related to httpd
getsebool -a | grep httpd

# Allow httpd to make outbound network connections (e.g. reverse proxy)
setsebool -P httpd_can_network_connect on

# Allow a service to bind a non-default port (e.g. Tomcat on 8081)
semanage port -l | grep http_port
```

```

semanage port -a -t http_port_t -p tcp 8081

# Search audit log for denials related to a service
ausearch -m avc -ts recent | grep httpd

# Explain WHY something was denied
audit2why < /var/log/audit/audit.log | tail -30

# Auto-generate a custom policy module to allow a denial
grep httpd /var/log/audit/audit.log | audit2allow -M mypolicy
semodule -i mypolicy.pp

```

HANDS-ON LAB: Diagnose and Fix an SELinux Denial on your Repo Server

1. Force a realistic denial: move repo content to a non-standard directory, e.g. /data/repo, without fixing context.
2. Point httpd at it and try `curl -I http://localhost/repo/` — expect a 403 error.
3. Confirm it is SELinux (not permissions) by checking context: `ls -Z /data/repo`.
4. Search the audit log: `ausearch -m avc -ts recent | grep httpd`.
5. Fix it permanently: `semanage fcontext -a -t httpd_sys_content_t "/data/repo(/.*)?"` then `restorecon -Rv /data/repo`.
6. Re-test with curl and confirm the 403 is gone — this is exactly the SELinux + Foreman/Katello pattern you hit in your homelab.

5.6 Cheat Sheet - SELinux

Command	Purpose	Example
<code>getenforce</code>	Show current mode	<code>getenforce</code>
<code>setenforce 0/1</code>	Change mode temporarily	<code>setenforce 0</code>
<code>sestatus</code>	Full SELinux status	<code>sestatus</code>
<code>/etc/selinux/config</code>	Permanent mode config file	<code>SELINUX=enforcing</code>
<code>ls -Z / ps -Z / id -Z</code>	View contexts	<code>ls -Z /var/www/html</code>
<code>restorecon -Rv</code>	Reset to default context	<code>restorecon -Rv /path</code>
<code>chcon -t</code>	Set temporary context	<code>chcon -t httpd_sys_content_t file</code>
<code>semanage fcontext -a</code>	Add persistent context rule	<code>semanage fcontext -a -t type "/path(/.*)?"</code>
<code>getsebool -a</code>	List all booleans	<code>getsebool -a grep ftp</code>
<code>setsebool -P</code>	Toggle boolean persistently	<code>setsebool -P httpd_can_network_connect on</code>
<code>semanage port -a</code>	Allow non-default port for a type	<code>semanage port -a -t http_port_t -p tcp 8081</code>
<code>ausearch -m avc</code>	Search audit log for denials	<code>ausearch -m avc -ts recent</code>
<code>audit2why</code>	Explain a denial in plain text	<code>audit2why < audit.log</code>
<code>audit2allow -M</code>	Generate custom policy module	<code>audit2allow -M mymod</code>
<code>semodule -i</code>	Install a custom policy module	<code>semodule -i mymod.pp</code>

6. Master Cheat Sheet - All Topics, One Reference

Package Management

```
dnf install -y <pkg> dnf remove -y <pkg> dnf update -y
dnf search <term> dnf info <pkg> dnf list installed
rpm -q <pkg> rpm -ql <pkg> rpm -qf <file>
dnf history list dnf history undo <id> rpm -V <pkg>
```

Repo Server / Client

```
createrepo_c <dir> createrepo_c --update <dir>
/etc/yum.repos.d/*.repo -> [name] baseurl= enabled=1 gpgcheck=0/1
reposync --repoid=<id> --download-path=<dir> -m
dnf clean all && dnf makecache dnf repolist all
```

Network

```
nmcli device status nmcli con show nmcli con up/down <name>
nmcli con mod <name> ipv4.addresses <ip>/<cidr> ipv4.gateway <gw> ipv4.dns "<dns>"
nmcli con mod <name> ipv4.method manual|auto hostnamectl set-hostname <fqdn>
ip a ip route ss -tulnp nmtui
```

Firewall (firewalld)

```
firewall-cmd --state firewall-cmd --list-all
firewall-cmd --permanent --add-service=<svc> firewall-cmd --permanent
--add-port=<p>/tcp
firewall-cmd --reload firewall-cmd --permanent --add-rich-rule='rule ...'
firewall-cmd --panic-on / --panic-off
```

SELinux

```
getenforce setenforce 0|1 sestatus
ls -Z <path> restorecon -Rv <path> chcon -t <type> <path>
semanage fcontext -a -t <type> "<path>(/.*)?" semanage port -a -t <type> -p tcp <port>
getsebool -a setsebool -P <bool> on|off
ausearch -m avc -ts recent audit2why < audit.log audit2allow -M <mod>
```

7. Exam-Style Quick Answers (Q&A Bank)

Short, exam/interview-ready answers you can recall quickly for theory questions on each topic.

Package Management

Q: What is the difference between RPM and DNF?

A: RPM installs/queries individual .rpm files with no automatic dependency resolution. DNF works against configured repositories and automatically resolves and downloads dependencies.

Q: How do you roll back a bad update?

A: Run 'dnf history list' to find the transaction ID, then 'dnf history undo <id>' to revert it.

Q: How do you find which package owns a given file?

A: Use 'rpm -qf /path/to/file' (for an already-installed file) or 'dnf provides /path/to/file' (works even if not yet installed).

Repo Server / Client

Q: What tool builds repository metadata?

A: createrepo_c scans a directory of RPM files and generates the repodata/ folder DNF needs to treat it as a repository.

Q: What are the three key fields in a .repo file?

A: baseurl (where the repo lives), enabled (1/0 to turn it on/off), and gpgcheck (whether to verify package signatures).

Q: Why might a client fail to reach an HTTP repo even though Apache is running fine?

A: Two common causes: the firewall is blocking port 80/443, or SELinux hasn't labeled the content directory with httpd_sys_content_t, so Apache is denied read access even though standard Unix permissions look correct.

Network

Q: What daemon manages networking on modern RHEL?

A: NetworkManager, controlled via nmcli, nmtui, or the Cockpit web console.

Q: How do you make a static IP change persist across reboot?

A: Modify the connection profile itself with 'nmcli con mod' (setting ipv4.method to manual plus addresses/gateway/dns) and bring it up with 'nmcli con up' - profile changes are saved to disk under /etc/NetworkManager/system-connections/ automatically.

Q: What's the difference between a device and a connection in NetworkManager terms?

A: A device is the physical/virtual interface (e.g. ens160). A connection is a saved configuration profile that gets applied to a device - one device can have multiple connection profiles, only one active at a time.

Firewall

Q: What's the difference between runtime and permanent firewall rules?

A: Runtime rules apply immediately but are lost on reload/reboot. Permanent rules (--permanent flag) are written to config but need '--reload' (or a matching non-permanent add) to also take effect now.

Q: What is a firewalld zone?

A: A named set of rules representing a trust level for a network interface, e.g. public, internal, trusted, dmz - each zone can allow different services/ports.

Q: How do you restrict a service to a single source IP or subnet?

A: Use a rich rule, e.g. firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.50.0/24" service name="ssh" accept'.

SELinux

Q: What are the three SELinux modes?

A: Enforcing (blocks and logs denials), Permissive (only logs, does not block - useful for troubleshooting), and Disabled (SELinux is off entirely).

Q: Why would a service fail with 'Permission denied' even though file permissions (rwx) look correct?

A: SELinux enforces a second, independent access-control layer based on security context (type). A process may be denied by policy even if standard Unix permissions would allow the action.

Q: What is the correct way to permanently fix a mislabeled directory, versus a quick temporary fix?

A: Quick/temporary: `chcon -t <type>` (lost on the next full relabel). Permanent/correct: `semanage fcontext -a -t <type> "/path(/.*)"?` followed by `restorecon -Rv /path`, so the custom rule survives future relabels.

Q: What is the purpose of an SELinux boolean?

A: A boolean is an on/off toggle for a pre-defined chunk of policy (e.g. `httpd_can_network_connect`), letting admins enable common exceptions without writing custom policy modules.

Compiled study reference covering Package Management, Repo Server/Client, Network, Firewall and SELinux - from RHCSA fundamentals through production/homelab-level practice.